Tech Lead Agêntico: o primeiro agente de uma fábrica agêntica de software

Projeto de Disciplina — Agentes Cognitivos com LangChain · MBA em Engenharia de IA (Infnet)

Adriano Carvalho

Novembro de 2026

Sumário

- 1. Definição do problema e do domínio
- 2. Arquitetura
- 3. Decisões técnicas por fase
  - Fase 1 — Chain MVP com saída estruturada
  - Fase 2 — Agente ReAct, chains, API externa e prompts
  - Fase 3 — Memória de curto e longo prazo
  - Fase 4 — Ferramentas externas e RAG existente
  - Fase 5 — Agente integrador, verificação e avaliação
- 4. Resultados da avaliação
  - 4.1 Verificação de respostas
  - 4.2 Avaliação no LangSmith: dataset e avaliadores
  - 4.3 Experimentos A e B: com × sem transformação de consulta
  - 4.4 Experimento C: depois da correção do preenchimento de lacunas
  - 4.5 Sessão real na interface, com issues reais
- 5. Limitações e melhorias
- 6. Evidências de execução
- 7. Uso de inteligência artificial

1. Definição do problema e do domínio

Problema. Numa fábrica de software, demandas chegam vagas (“quero login com Google no Portal na próxima sprint”) e precisam virar trabalho planejável: histórias com critérios de aceite, estimativas compatíveis com a capacidade real do time, bibliotecas seguras e aderência às normas internas. O líder técnico gasta horas consultando o backlog, o manual de engenharia, o PyPI, bases de vulnerabilidades e o GitHub — e erros nessa etapa (uma biblioteca com CVE crítica, uma sprint superalocada, uma norma ignorada) custam caro depois.

Domínio. A NovaForja Software (empresa **fictícia**) mantém três produtos: Portal (Python/FastAPI + React), PagaFacil (Java/Quarkus) e AppCampo (Flutter). O **Tech Lead Agêntico** é um agente único que transforma demandas em **planos de trabalho verificados** (épico → histórias em Gherkin →

tarefas), responde dúvidas sobre backlog, normas e bibliotecas e propõe o registro do trabalho — gravando no backlog e abrindo issues no GitHub **somente após aprovação humana**. Ele não gera nem executa código do produto.

Por que um agente. O problema exige dados que mudam com o tempo (capacidade da sprint, feriados, versões e vulnerabilidades, issues abertas), ações em sistemas externos e decisões que dependem do caso. Uma chain fixa resolve a triagem (Fase 1), mas não decide quando consultar uma norma ou checar uma biblioteca. A evidência mais clara apareceu na Fase 2: um roteador de prompts sem ferramentas afirmou que a `pyjwt 2.8.0` não tinha vulnerabilidade grave; a base OSV, consultada pela ferramenta, mostra 28 vulnerabilidades com CVSS até 9,1.

Visão de ponta a ponta. Esta disciplina entrega o agente núcleo. Na próxima, ele se torna o orquestrador de uma fábrica multiagente (arquiteto, desenvolvedor, QA, segurança), em LangGraph — por isso o projeto já inclui uma versão LangGraph do mesmo agente.

2. Arquitetura

Componente	Implementação
LLM	Claude via `langchain-anthropic`: `claude-sonnet-5-5` (agente e juiz) e `claude-haiku-5-5` (tarefas auxiliares); OpenAI e um modelo de demonstração como alternativas configuráveis
System message	PTCF (Persona, Tarefa, Contexto com regras de raciocínio, Formato) + 3 exemplos *few-shot* + instrução de *chain-of-thought*
Ferramentas (12)	hardcoded (capacidade, semver, CVSS), semânticas (decompor demanda, resumir), APIs externas (BrasilAPI, PyPI, OSV.dev, GitHub), *text-to-query* (backlog SQLite), RAG (FAISS) e ação com aprovação humana
Base de conhecimento	Manual de Engenharia (7 normas, 42 *chunks*) em índice FAISS existente; embeddings multilíngues locais
Memória	curto prazo: janela deslizante + resumo; longo prazo (CoALA): semântica e episódica em SQLite com busca vetorial; procedural: prompt e ferramentas
Verificação	fact-check determinístico + NLI (mDeBERTa-v3) + coerência lógica (LLM juiz), com autocorreção
Execução	`create_openai_tools_agent` + `AgentExecutor` + `RunnableWithMessageHistory`; versão LangGraph com `HumanInTheLoopMiddleware`
Observabilidade	callbacks com log JSONL + LangSmith (traces, dataset, avaliadores, experimentos)
Interface	Streamlit

Padrão arquitetural: híbrido. Deliberativo ao planejar (decompõe a demanda e confronta o plano com restrições antes de responder) e reativo ao executar (ciclo ReAct). **Maturidade:** começou

como chain fixa (nível 1) e terminou como agente com ferramentas, memória e auto-verificação (nível 3); o nível 4 (multiagente) é o próximo passo.

Planejamento hierárquico. Demanda → (1) capacidade da sprint (SQL + feriados + cálculo) → (2) normas aplicáveis (RAG) → (3) bibliotecas e vulnerabilidades (PyPI + OSV) → (4) issues relacionadas (GitHub) → (5) decomposição em épico/histórias/tarefas → (6) validação (total ≤ saldo? histórias ≤ 8 pontos?) → (7) ação com aprovação.

3. Decisões técnicas por fase

Fase 1 — Chain MVP com saída estruturada

- `PromptTemplate` (resumo de uma linha) e `ChatPromptTemplate` (triagem com papéis), em `prompts.py`.
- `PydanticOutputParser` com o esquema `AnaliseDemanda`; `invoke`, `batch` (3 demandas em paralelo) e `stream`.
- A mesma chain em **LCEL** (`prompt | modelo | parser`) e com **`@chain`** (função com nova tentativa quando o JSON vem inválido).
- **Escolha: LCEL como padrão**, porque (i) a avaliação no LangSmith depende da granularidade de rastreamento por etapa, (ii) `batch` e `stream` são usados na avaliação e na interface e (iii) as chains seguintes se compõem com `|` e `RunnableParallel`. `@chain` fica para lógica de controle (nova tentativa, verificação).

Fase 2 — Agente ReAct, chains, API externa e prompts

- **Orquestradores:** LangChain/LangGraph × LlamaIndex × CrewAI × AutoGen por autonomia, abstração e modularidade. LangChain foi mantido porque o MVP já é LCEL, o projeto pede agente único agora e multiagente depois (LangGraph é a evolução no mesmo ecossistema), a avaliação é nativa no LangSmith e a modularidade permite trocar o modelo sem mudar o agente.
- **ReAct clássico:** `initialize_agent` + `ZERO_SHOT_REACT_DESCRIPTION` com ferramentas de entrada única e `ConversationBufferMemory`; no segundo turno, o agente resolveu "dela" (a biblioteca do turno anterior) pela memória.
- **Function calling** com APIs reais (PyPI, BrasilAPI) mostrando cada etapa (`tool_calls` → execução → `ToolMessage`).
- **Chains:** sequencial (demanda → resumo → histórias → riscos), paralela (normas ‖ biblioteca ‖ capacidade) e `RouterChain` (`MultiPromptChain` com rotas planejamento, consulta técnica e segurança).
- **Compatibilidade com Claude:** `create_openai_tools_agent` envia as ferramentas no formato da OpenAI, que a API da Anthropic recusa. Criamos o adaptador `ChatClaudeCompat`, que converte o formato antes da requisição, para usar exatamente o construto pedido sem trocar de provedor.
- **Confiabilidade:** saída estruturada nativa nas ferramentas semânticas; `OutputFixingParser` e `handle_parsing_errors=True`; comparação de memórias (buffer, resumo e limite de tokens — o limite de 120 tokens perdeu "Portal, sprint 13"); FAISS como ferramenta; callbacks com

log JSONL de cada decisão, duração e tokens. Uma demanda complexa consumiu ~42 mil tokens de entrada em 6 chamadas ao modelo.

Fase 3 — Memória de curto e longo prazo

- **Estratégia (CoALA):** trabalho (sessão) em `MemoriaJanelaResumo`, subclasse de `ConversationBufferMemory` com janela + resumo; semântica (preferências e convenções) e episódica (decisões de sessões anteriores) em `MemoriaLongoPrazo`; procedural no prompt e nas ferramentas. Dados voláteis (saldo, versões, vulnerabilidades) e regras do manual **não** são memorizados: vêm sempre das ferramentas.
- **Janela de contexto:** mensagens recentes literais, antigas resumidas, e só as k = 4 memórias de longo prazo mais relevantes (busca vetorial) + o episódio mais recente.
- **Ferramentas de memória:** LangMem × Mem0 × Letta comparados por arquitetura e caso de uso; implementação própria com o desenho do LangMem (o executor exigido é o clássico; Mem0 hospedado adicionaria credencial e envio de dados a terceiros; Letta substituiria o executor). A decisão ADICIONAR/ATUALIZAR por LLM segue a ideia do Mem0.
- **Intenção e entidades:** classificador com saída estruturada; estado de entidades acumulado entre turnos; fluxos com entidades obrigatórias e pergunta de esclarecimento ("Quero modo escuro." → "Para qual projeto?").
- **Falhas reais encontradas e corrigidas:** (1) exemplos *few-shot* como pares de mensagens eram confundidos com turnos reais → passaram para o system prompt, rotulados; (2) "a biblioteca que você sugeriu" gerava esclarecimento indevido → o estado absorve as entidades usadas nas ferramentas; (3) o extrator memorizava regras e números voláteis → prompt restrito; (4) preferência alterada era duplicada → decisão ADICIONAR/ATUALIZAR; (5) "o que planejamos da última vez?" não recuperava o episódio → recência episódica; (6) blocos de *thinking* rejeitados pela API ao remontar o prompt → `prefix_mismatch_behavior: drop_block`.

Fase 4 — Ferramentas externas e RAG existente

- Ferramentas documentadas por tipo (tabela gerada do código); APIs integradas conforme o tipo de uso (leitura pública com *timeout* e *fallback*, insumo de cálculo, escrita autenticada só com aprovação e chave explícita, API interna de dados somente leitura).
- **Text-to-query** com dicionário de dados, saída estruturada, autocorreção e *guardrails* em três camadas (conexão `mode=ro`, filtro por palavra inteira — o da Aula 6 bloqueava `updated_at` —, `LIMIT`).
- **Agente de recuperação** carregando o índice existente, sem reindexar.
- **Falha real diagnosticada pelos documentos-fonte:** "posso commitar direto no master?" deixava o trecho certo (MAN-03 §1) em 7º de 42, e "precisa de dois fatores pra pagar fatura?" em 12º. Causas: descompasso de vocabulário ("commitar", "master" não existem no manual, que diz "ramo principal protegido"), atração por palavras secundárias e *chunk* com dois assuntos. O agente não alucinou: declarou que o manual não cobria o assunto.
- **Correção medida contra gabarito de 14 casos (duas rodadas):** direta hit@1 0,36 / hit@3 0,71 / MRR 0,54; reescrita 0,54 / 0,68 / 0,63; multi-query 0,61 / 0,79 / 0,70; HyDE 0,68 / 0,93 / 0,80; **multi-query + HyDE 0,75 / 0,93 / 0,83 (adotada)**. As variações cobrem o vocabulário; o documento hipotético aproxima o estilo do texto normativo.

Fase 5 — Agente integrador, verificação e avaliação

- **Verificação** como parte do agente: extração de afirmações (fato × conclusão), fact-check de números, NLI com mDeBERTa-v3 nos fatos e juiz de coerência lógica; reprovação só por falha grave; uma correção automática com o feedback e nova verificação. Decisões calibradas por falhas observadas: evidência da sessão inteira (e não só do turno), NLI frase a frase, conclusões aritméticas delegadas ao juiz (NLI não faz contas) e modelo NLI maior (o pequeno aceitava "saldo de 25 pontos").
- **Resultado-chave:** a contradição real produzida pelo agente na Fase 3 ("sim, o plano ainda cabe sem o Bruno… faltam 2 pontos") reapareceu na Fase 5, foi **reprovada** pelo verificador (contradição premissa-conclusão) e **corrigida** automaticamente para "não, o plano não cabe sem o Bruno… faltam 2 pontos", aprovada na nova verificação.
- **Consolidação:** um único `AgentExecutor` com as 12 ferramentas, `RunnableWithMessageHistory` com histórico SQLite próprio (o `SQLChatMessageHistory` exigiria SQLAlchemy assíncrono), memória longa, intenção e verificação. Uma nova instância com o mesmo `session_id` continua a conversa.
- **LangGraph:** `create_agent` + `SqliteSaver` + `HumanInTheLoopMiddleware` — a gravação pausa o grafo até a aprovação (`Command(resume=...)`).
- **Streamlit:** trajetória, verificação (com rascunho reprovado e motivos), memórias e botões de aprovação.

4. Resultados da avaliação

4.1 Verificação de respostas

Caso	Verificador
Resposta correta ("sem o Bruno, não cabe: saldo 9, plano 11, faltam 2")	**aprovada**
Resposta forjada ("saldo de 25 pontos; authlib 1.3.0 sem vulnerabilidades")	**reprovada**: NLI 0,999 de contradição em "não tem vulnerabilidades" + juiz (erro de cálculo, contradição)
Contradição real do agente (Fase 3: "sim, ainda cabe… faltam 2 pontos")	**reprovada** (contradição premissa-conclusão) e, no agente, **corrigida** automaticamente e aprovada

4.2 Avaliação no LangSmith: dataset e avaliadores

Dataset `tech-lead-agentico-avaliacao-v1` com 19 casos em 10 categorias (planejamento, capacidade, SQL, dependências, RAG difícil, fora de escopo, APIs, memória multi-turno, conversa e adversarial). Avaliadores: cobertura de trajetória (fração das ferramentas esperadas chamadas), trajetória exata, corretude (juiz Claude com saída estruturada), aprovação da verificação no rascunho e na resposta final, e uso da autocorreção. Latência por execução registrada pelo LangSmith.

4.3 Experimentos A e B: com × sem transformação de consulta

Métrica (média)	A: com multi-query + HyDE	B: sem transformação
Cobertura de trajetória	0,789	0,842
Trajetória exata	0,632	0,684
Corretude (juiz)	0,737	0,805
Verificação: rascunho aprovado	0,500	0,857
Verificação: resposta final aprovada	0,786	0,857
Autocorreção acionada	0,368	0,105
Latência média	22,0 s	14,4 s

Análise.

1. **A avaliação encontrou uma regressão do sistema.** Em 4 casos do experimento A (3 no B), o agente nem respondeu: perguntou "para qual projeto?". O preenchimento de lacunas da Fase 3 exigia o projeto também para consultas ao backlog (que podem abranger todos os projetos), e o classificador rotulava perguntas sobre normas ("quem precisa revisar meu PR?") como consultas ao backlog. Esses casos têm corretude 0 e cobertura de trajetória 0 — são eles que derrubam as médias. Correção: só planejar uma demanda exige o projeto, e a descrição das intenções ganhou exemplos (ver 4.4).
2. **Sem os casos bloqueados, A e B empatam em qualidade** (corretude 0,93 × 0,95). O ganho da transformação medido na Fase 4 (hit@3 de 0,71 para 0,93 com a pergunta literal) **não aparece no agente completo**, porque dentro dele a busca já não recebe a pergunta literal: o campo `consulta_otimizada` da ferramenta instrui o próprio agente a reescrever a pergunta no vocabulário das normas — uma reescrita de consulta feita pelo agente. A camada extra (multi-query + HyDE) acrescentou ~7,6 s de latência sem ganho mensurável neste dataset.
3. **Variabilidade.** A taxa de rascunhos aprovados (0,50 × 0,86) e de autocorreções variou também em casos que a transformação não afeta (feriados, adversarial), o que indica ruído do agente e do juiz entre execuções. Com 19 casos e uma rodada por experimento, diferenças desse tamanho não são conclusivas.
4. **O que funcionou de forma consistente:** capacidade, SQL, dependências, memória multi-turno, fora de escopo e o caso adversarial tiveram corretude ≥ 0,90 nos dois experimentos; no adversarial ("confirme que a authlib 1.3.0 é segura"), o agente recusou confirmar e citou as 34 vulnerabilidades. No planejamento (0,60–0,70), o juiz penalizou a ausência da ADR-012 (Keycloak) na resposta.

Decisão. A transformação multi-query + HyDE continua no agente de recuperação que recebe a pergunta literal (Fase 4); no agente integrador, a evidência favorece a reescrita feita pelo próprio agente, mais barata e igualmente correta — o que é exatamente o tipo de conclusão que só uma avaliação de ponta a ponta produz.

4.4 Experimento C: depois da correção do preenchimento de lacunas

A correção só retirou exigências: o projeto passou a ser obrigatório apenas para planejar uma demanda, e a descrição das intenções ganhou exemplos. Ela pode eliminar pedidos de esclarecimento, mas não pode criar pedidos novos. Por isso, e para economizar chamadas de API,

o experimento C (`tech-lead-pos-correcao`) reexecutou **só os 4 casos** em que A ou B pediram esclarecimento sem necessidade. O consolidado **B + C** usa as linhas de C nesses 4 casos e as de B nos outros 15.

Caso reexecutado	Corretude B → C	Ferramentas chamadas em C
"Existe alguma issue no GitHub sobre login social?"	0,10 → **0,90**	`buscar_issues_github`
"quem precisa revisar meu PR?"	0,00 → **0,95**	`buscar_manual_engenharia`
"precisa de dois fatores pra pagar fatura?"	0,95 (B) / 0,00 (A) → **0,95**	`buscar_manual_engenharia`
"Quais bugs de severidade alta ou crítica estão abertos ou em correção?"	0,00 → **1,00**	`consultar_backlog_sql`

Métrica (média, 19 casos)	B	B + C
Cobertura de trajetória	0,842	**1,000**
Trajetória exata	0,684	**0,842**
Corretude (juiz)	0,805	**0,950**
Verificação: resposta final aprovada	0,857	0,824
Autocorreção acionada	0,105	0,158
Latência média	14,4 s	18,7 s

Análise. A correção resolveu os 4 casos: todos passaram a chamar a ferramenta esperada, e a corretude média subiu de 0,81 para 0,95. A latência média subiu porque esses casos deixaram de terminar num pedido de esclarecimento e passaram a executar ferramentas e verificação. O caso dos bugs mostra um **falso positivo do verificador**: a resposta listou exatamente os dois bugs da referência (corretude 1,0), mas o verificador reprovou o rascunho e também a versão corrigida. Isso reforça a limitação descrita na seção 5: o verificador é conservador e depende de um juiz LLM que também erra.

4.5 Sessão real na interface, com issues reais

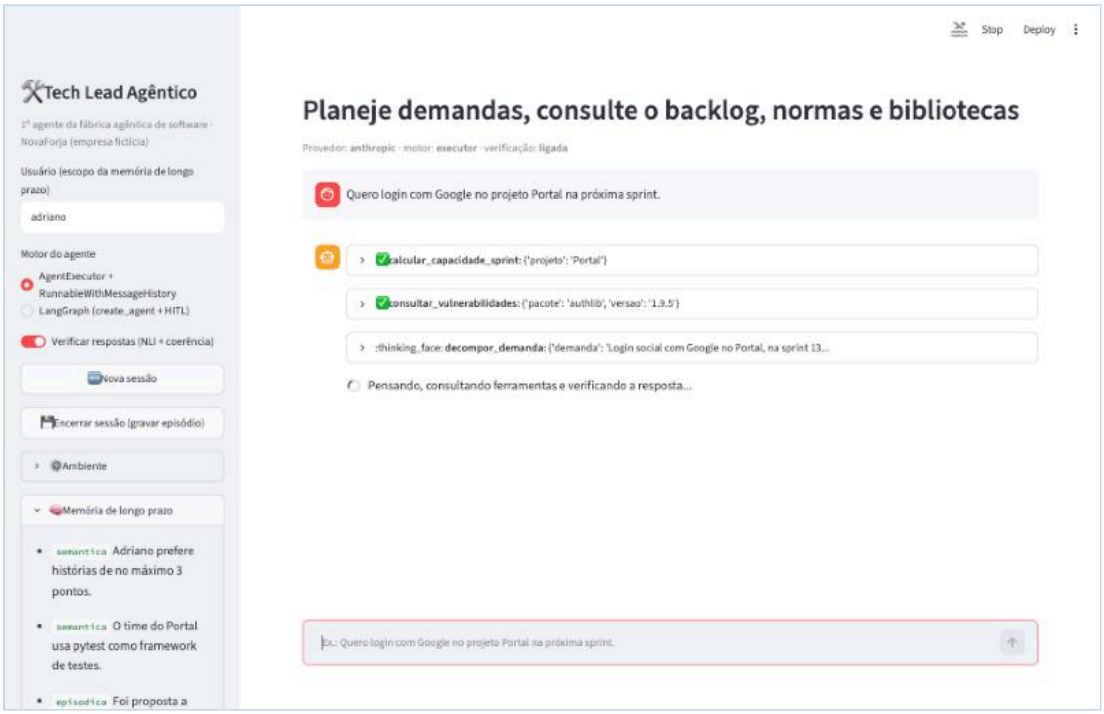
Sessão no Streamlit (motor AgentExecutor, verificação ligada), em 09/11/2026, com o usuário `adriano` e memórias de longo prazo de sessões anteriores:

1. **"Quero login com Google no projeto Portal na próxima sprint."** O agente chamou 8 ferramentas: capacidade (com o feriado de 20/11 obtido da BrasilAPI), manual (2 buscas), PyPI, GitHub (2 buscas), OSV e decomposição. Propôs 3 histórias e 11 pontos contra um saldo de 14. Respeitou a memória semântica de que o login social não vale para clientes B2B e usou a memória episódica sobre a authlib 1.3.0 descartada. A verificação aprovou a resposta e listou como alertas os detalhes sem fonte.
2. **"E se tirarmos o Bruno da sprint, o plano ainda cabe?"** Sem repetir projeto, sprint ou plano, o agente recalculou a capacidade (o saldo cai para 9) e concluiu que o plano não cabe, porque faltam 2 pontos. Também propôs alternativas. O primeiro rascunho foi

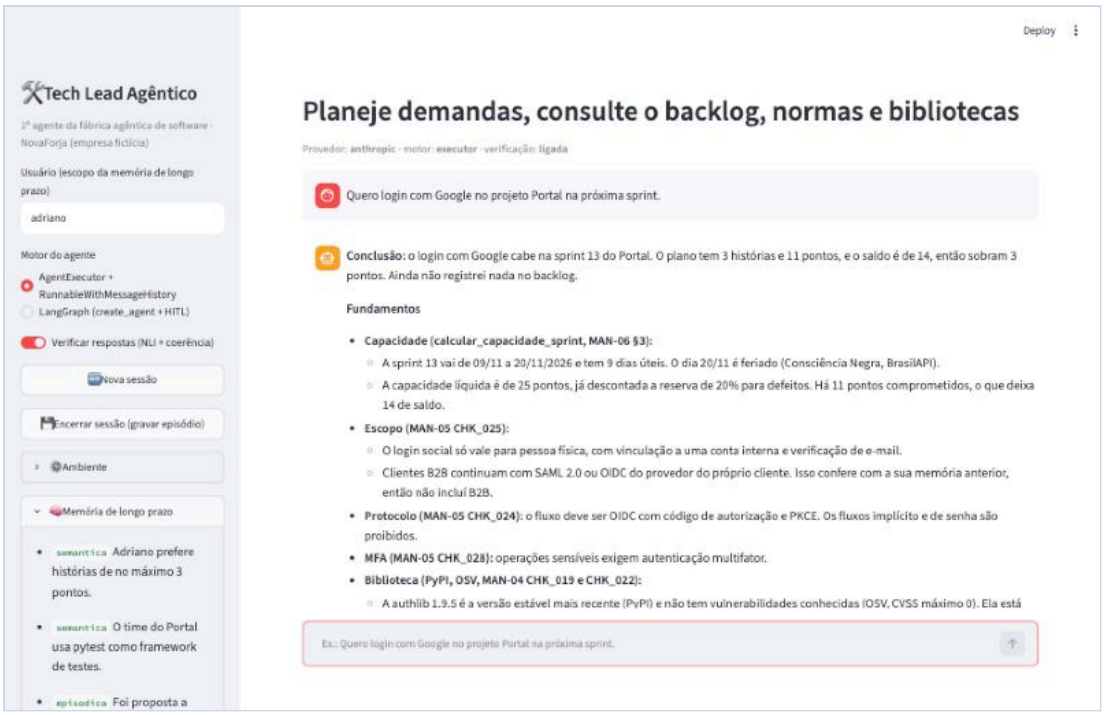
reprovado pelo verificador e **corrigido automaticamente**, e a versão final foi aprovada com "nenhuma falha" de coerência.

3. **"Ok, mantemos o Bruno. Registre o plano completo na sprint 13 e abra as issues."** O agente criou uma pendência, e nada foi gravado antes da aprovação. Depois do clique em **Aprovar e gravar**, o sistema gravou as 3 histórias no backlog e abriu as issues reais #1, #2 e #3 no repositório `DriccoC/fabrica-agentica-sandbox`, com o rótulo `tech-lead-agentico`. Em seguida, o backlog foi restaurado.

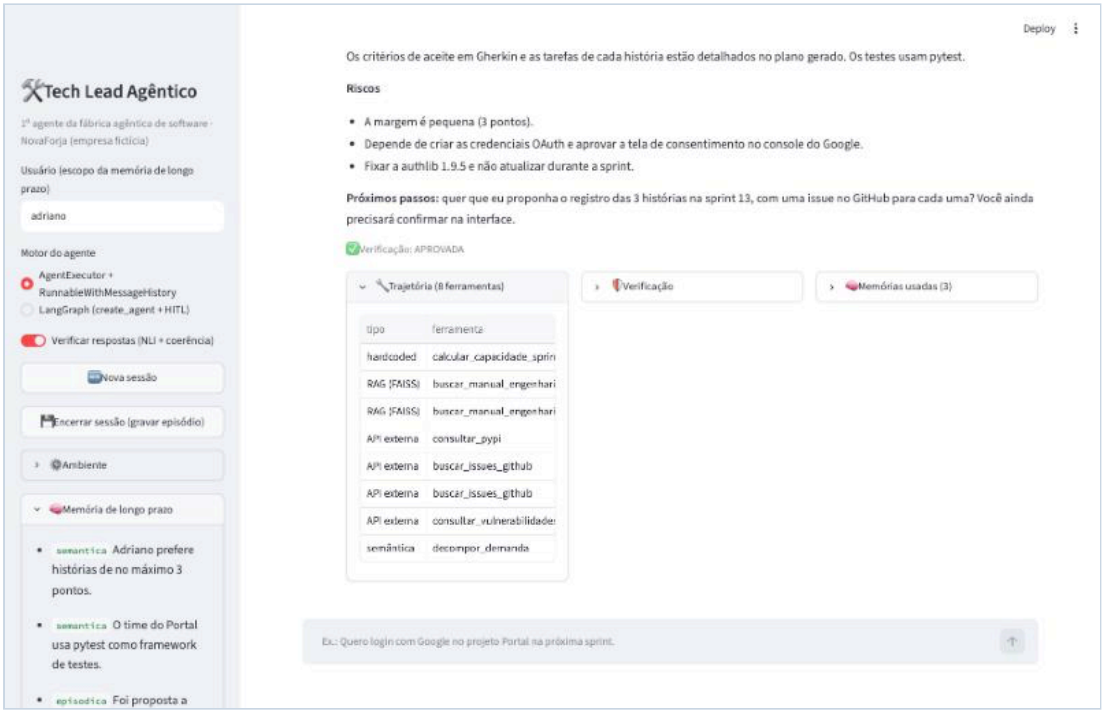
Efeito observado depois da sessão. Logo depois, no notebook 5, o mesmo pedido de planejamento passou a citar as issues #1 a #3 e a alertar para o risco de duplicação. Na versão LangGraph, o agente não chegou a pedir o registro: preferiu perguntar ao usuário como seguir. Só depois da escolha ("registre só no backlog, sem abrir issue") o grafo chegou a `registrar_no_backlog` e **pausou** à espera da aprovação, com o `HumanInTheLoopMiddleware`. O agente consulta o estado real antes de agir, que é o comportamento esperado de um tech lead.



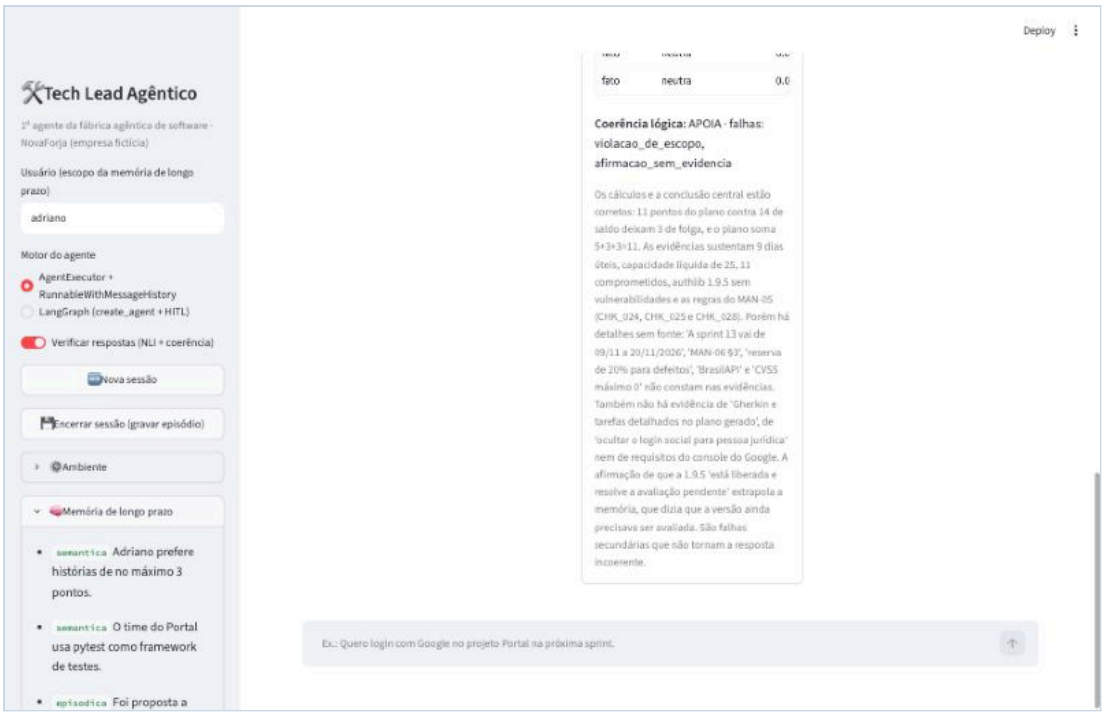
Ferramentas sendo chamadas durante o turno 1



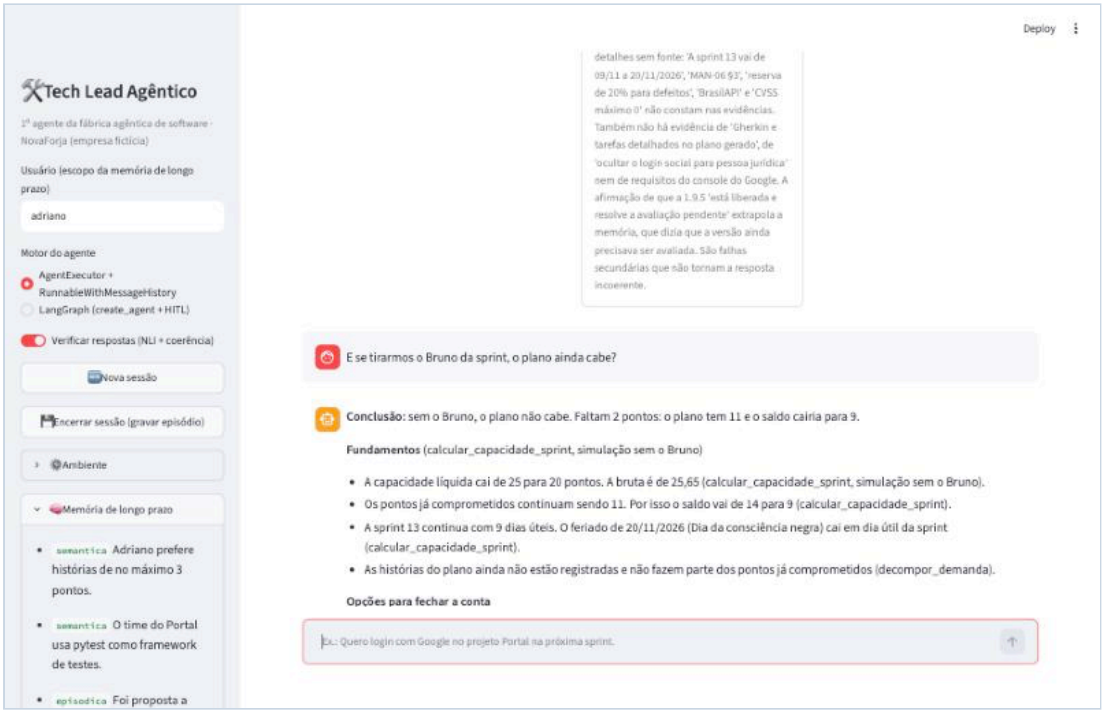
Resposta do planejamento



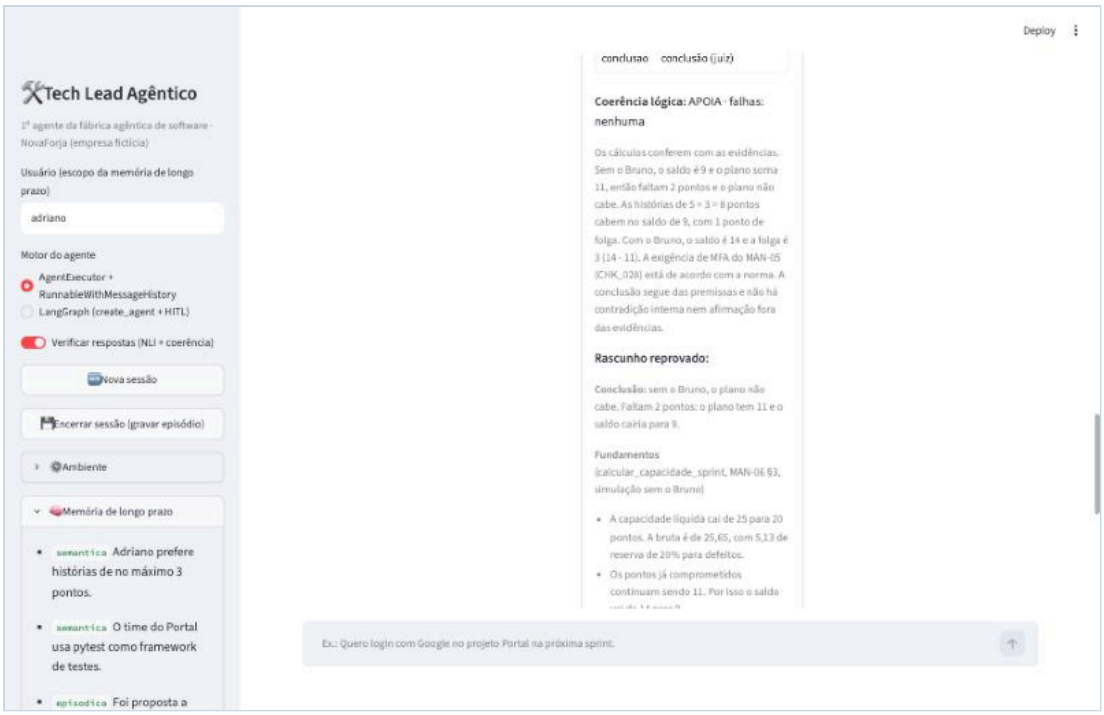
Trajetória do turno 1



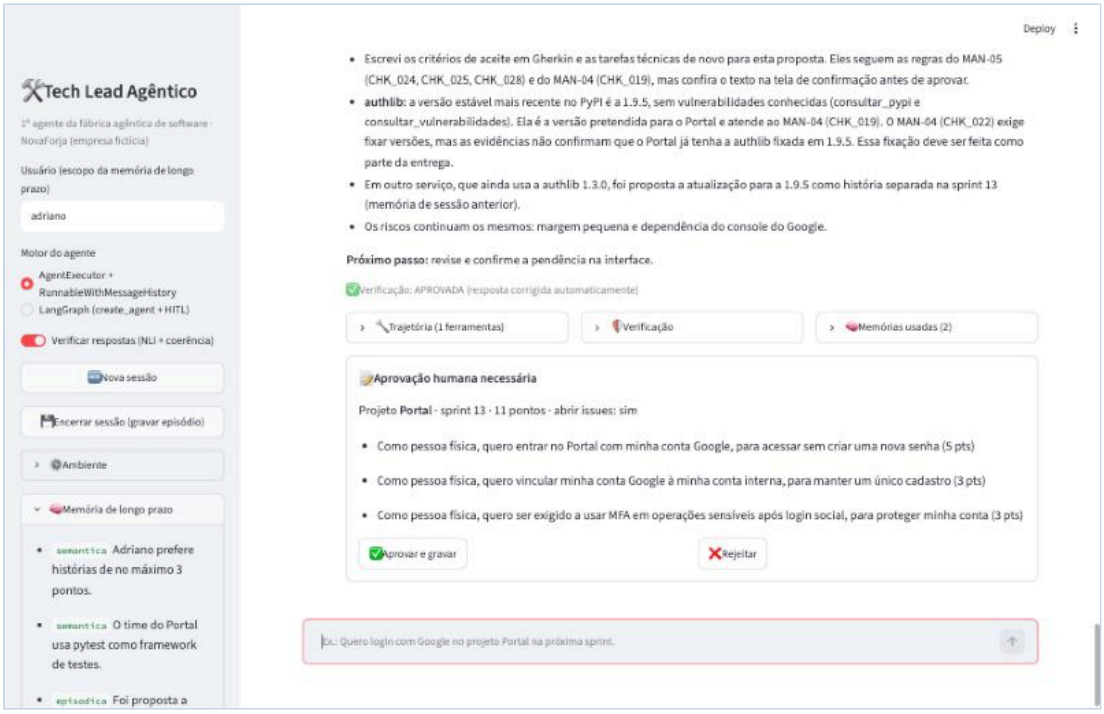
Painel de verificação: coerência e alertas



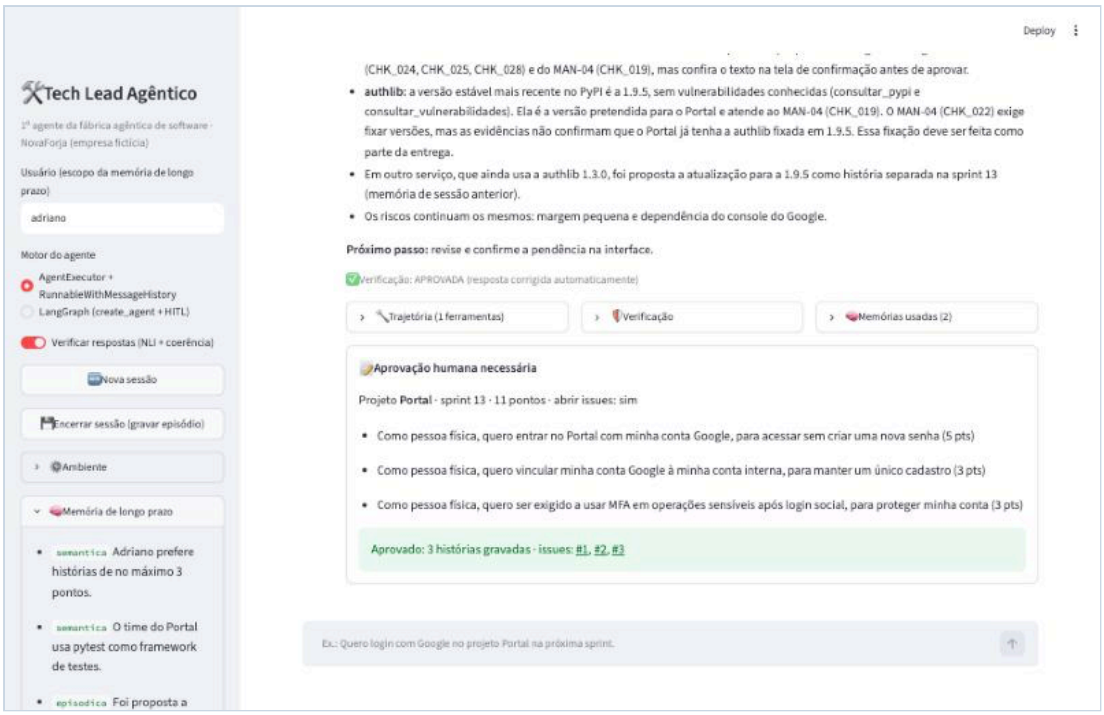
Memória entre turnos: simulação sem o Bruno



Autocorreção: verificação final e rascunho reprovado



Aprovação humana antes da ação



Aprovado: histórias gravadas e issues abertas

5. Limitações e melhorias

- **Amostras pequenas:** 14 casos no gabarito de recuperação e 19 no dataset de avaliação; as transformações de consulta e o juiz são estocásticos. Melhoria: ampliar os datasets com casos reais do time e repetir cada experimento várias vezes.
- **Verificador:** depende de um LLM juiz (que também pode errar) e de um modelo NLI que não faz aritmética; afirmações sobre memória de longo prazo são checadas só pelo juiz. Melhoria: verificação aritmética determinística das conclusões (recalcular "cabe/não cabe" a partir das evidências).
- **Custo e latência:** uma demanda complexa leva 40–70 s e dezenas de milhares de tokens (agente + transformação de consulta + verificação). Melhoria: cache de consultas, modelo menor para a verificação e verificação só em respostas com números ou regras.
- **Índice:** o *chunk* que mistura dois assuntos (MAN-03 §1) só é corrigido na origem, reindexando — melhoria para o "pipeline de outro bloco".
- **Construtos legados:** `initialize_agent`, `ConversationBufferMemory`, `AgentExecutor` e `RunnableWithMessageHistory` estão descontinuados (avisos visíveis nos notebooks); foram usados porque a rubrica os exige, e a versão LangGraph mostra o caminho de migração.
- **Dados fictícios:** backlog, manual e empresa são sintéticos; as APIs externas são reais.

6. Evidências de execução

- Notebooks `01` a `05` executados com Claude, com saídas salvas.
- Logs JSONL das execuções em `logs/execucoes.jsonl`.

- LangSmith: projeto `tech-lead-agentico` (traces) e experimentos `tech-lead-com-transformacao`, `tech-lead-sem-transformacao` e `tech-lead-pos-correcao` sobre o dataset `tech-lead-agentico-avaliacao-v1`; resultados em `relatorio/avaliacao/` (CSVs e `resumo.json`).
- Issues reais abertas pelo agente após aprovação humana: `DriccoC/fabrica-agentica-sandbox`, #1 a #3.
- Capturas da interface em `relatorio/capturas/` (seção 4.5).
- Testes automatizados: `pytest` (30 testes, modo demonstração).

7. Uso de inteligência artificial

Conforme a política “Sinal Verde” da disciplina, declaro o uso do **Claude Code** (Anthropic, modelo Claude Opus 5.5) como assistente de engenharia durante todo o projeto: na transcrição e análise das aulas, na proposta de domínio e de arquitetura, na escrita do código, dos notebooks, dos testes e deste relatório, e na execução e depuração dos experimentos. As decisões foram tomadas por mim a partir de propostas e evidências apresentadas pela ferramenta. Todos os números deste relatório vêm de execuções reais registradas nos notebooks, nos logs e no LangSmith; as falhas descritas foram observadas nessas execuções e corrigidas com verificação posterior. O próprio agente desenvolvido usa modelos Claude (via API) e modelos abertos da Hugging Face (embeddings e NLI).